

# LiteLLM + Huawei MaaS (GLM-5.1) + Codex

A self-hosted proxy that lets OpenAI Codex Desktop use Huawei ModelArts MaaS models (GLM-5.1, GLM-5, DeepSeek) through the Responses API.

Tested with LiteLLM `v1.83.14-stable.patch.3` · Docker Compose v5.2 · Ubuntu 22.04 · Region `ap-southeast-1` · Plugin `responses_tool_guard`

**What you get:** Codex Desktop talks to `http://YOUR-SERVER:4000` using the OpenAI Responses API. LiteLLM strips the unsupported `namespace / web_search` tools, bridges `/responses` → `/chat/completions`, and forwards to Huawei MaaS. No more `400 "request param validation error, 'function'"`.

## 0 Architecture

```
Codex Desktop —/v1/responses→ LiteLLM Proxy (:4000)
| 1. responses_tool_guard plugin
|   strips namespace / web_search tools
|   (flattens namespace sub-tools → functions)
| 2. use_chat_completions_api: true
|   bridges /responses → /chat/completions
| 3. source patches (proxy_server.py, utils.py)
|   adapt the streaming path for MaaS
|
|   ↓
Huawei ModelArts MaaS (GLM-5.1 / GLM-5 / DeepSeek)
https://api-ap-southeast-1.modelarts-maas.com/openai/v1

Sidecar services: PostgreSQL (:5432) · Prometheus (:9090) · Grafana (:3000)
```

## 1 Prerequisites

- A Linux server (Ubuntu 22.04+ recommended) with **root/sudo** access and at least 2 vCPU / 4 GB RAM.
- **Docker Engine** and **Docker Compose v2+** installed.
- A **Huawei Cloud account** with ModelArts MaaS enabled and an API Key for the `ap-southeast-1` region.
- OpenAI **Codex Desktop** (or Codex CLI) on your workstation.
- Ports `4000` open on the server firewall (plus `3000` / `9090` if you want Grafana/Prometheus).

## 2 Get your Huawei MaaS API key

1. Log in to the Huawei Cloud console → ModelArts → MaaS.
2. Create an **API Key** in region `ap-southeast-1` (Asia Pacific Southeast).

3. Note the key — it starts with `sk-`. You'll store it as `HUAWEI_MAAS_API_KEY_0`.
4. The API base for this region is:  
`https://api-ap-southeast-1.modelarts-maas.com/openai/v1`

## 3 Deploy the LiteLLM stack

### 3.1 — Put the project on the server

Create a project directory (e.g. `/home/LiteLLM-Huawei-MaaS-Proxy`) containing the `docker-compose.yml`, the `assets/config/` tree, and the `patches/` tree. The layout used in this guide is:

```
/home/LiteLLM-Huawei-MaaS-Proxy/
├── docker-compose.yml
├── .env
├── assets/config/
│   ├── litellm_config.yaml
│   ├── custom_callbacks.py
│   ├── rolling_budget_hook.py
│   ├── responses_tool_guard.py    ← plugin (Step 5)
│   ├── prometheus.yml
│   └── grafana/provisioning/
├── patches/
│   ├── proxy_server.py          ← streaming-path patches
│   ├── utils.py
│   └── README.md
└── adapter/                    ← optional Anthropic adapter
```

### 3.2 — Create the `.env` file

```
# — Huawei MaaS —
HUAWEI_MAAS_API_BASE=https://api-ap-southeast-1.modelarts-maas.com/openai/v1
HUAWEI_MAAS_API_KEY_0=sk-your-huawei-key-here

# — LiteLLM database —
DB_PASSWORD=change-this-strong-password
LITELLM_MASTER_KEY=sk-your-litellm-master-key # pick any strong sk-... string

# — Optional: OpenRouter (for vision models) —
OpenRouter_API_KEY=sk-or-...

# — Grafana —
GRAFANA_PASSWORD=change-this
```

**Security:** `LITELLM_MASTER_KEY` is the admin key for the proxy — make it a long random string. Never commit `.env` to git.

### 3.3 — Start the stack

```
cd /home/LiteLLM-Huawei-MaaS-Proxy
docker compose up -d
```

Check that all containers are healthy:

```
docker ps --format "{{.Names}}\t{{.Status}}"
```

You should see `litellm_proxy`, `litellm_pg_db`, `litellm_prometheus`, `litellm_grafana` all `Up ... (healthy)`.

## 4 Configure the models

Edit `assets/config/litellm_config.yaml`. The two things that make Huawei MaaS work with Codex are **the model group** and the `use_chat_completions_api` bridge.

```
model_list:
- model_name: glm-5.1
  litellm_params:
    model: openai/glm-5.1
    api_base: os.environ/HUAWEI_MAAS_API_BASE
    api_key: os.environ/HUAWEI_MAAS_API_KEY_0
    use_chat_completions_api: true # ← bridges /responses → /chat/completions
    tpm: 500000
    rpm: 30
  model_info:
    max_tokens: 198000
    max_input_tokens: 192000
    max_output_tokens: 128000
    input_cost_per_token: 0.000001078
    output_cost_per_token: 0.000003774

# Repeat the same pattern for every other Huawei model you want:
# glm-5, deepseek-v4-pro, deepseek-v4-flash, deepseek-v3.2
# Each MUST have use_chat_completions_api: true.

litellm_settings:
  num_retries: 3
  request_timeout: 600
  stream_timeout: 60
  drop_params: True
  set_verbose: False
  callbacks:
    - "prometheus"
    - custom_callbacks.my_prometheus_logger
    - rolling_budget_hook.proxy_handler_instance
    - responses_tool_guard.proxy_handler_instance # ← added in Step 5
```

**Important:** Add `use_chat_completions_api: true` to **every** Huawei model group (including fallbacks). Without it, LiteLLM forwards `/responses` to the upstream, which returns `404 APIG.0101`.

## 5 Install the responses\_tool\_guard plugin

This plugin strips the `namespace` and `web_search` tools that Codex sends but Huawei MaaS rejects.

Repo: <https://github.com/Zefirox/litellm-responses-tool-guard>

### 5.1 — Download the plugin

```
cd /home/LiteLLM-Huawei-MaaS-Proxy
curl -fsSL -o assets/config/responses_tool_guard.py \
  https://raw.githubusercontent.com/Zefirox/litellm-responses-tool-guard/main/responses_tool_guard.py
```

### 5.2 — Mount it in docker-compose.yml

Add this line to the `litellm` service `volumes:` block:

```
- ./assets/config/responses_tool_guard.py:/app/responses_tool_guard.py:ro
```

### 5.3 — Add the plugin environment variables

Add to the `litellm` service `environment:` block:

```
RTG_FLATTEN_NAMESPACES: "true"
RTG_AFFECTED_MODELS: "glm-5.1,glm-5,deepseek-v4-pro,deepseek-v4-flash,deepseek-v3.2"
```

| Env var                             | Default            | Meaning                                                                                                                                                     |
|-------------------------------------|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>RTG_FLATTEN_NAMESPACES</code> | <code>false</code> | <code>true</code> = promote namespace sub-tools (e.g. <code>spawn_agent</code> ) to individual <code>function</code> tools. <code>false</code> = drop them. |
| <code>RTG_AFFECTED_MODELS</code>    | empty (all)        | Comma-separated model group names to guard. Empty = apply to every model.                                                                                   |

### 5.4 — Register the callback

Add `responses_tool_guard.proxy_handler_instance` to `litellm_settings.callbacks` (shown in Step 4).

## 6 Restart & verify

```
cd /home/LiteLLM-Huawei-MaaS-Proxy
docker compose up -d litellm          # pick up the new mount + env
docker restart litellm_proxy          # reload the config file

# wait for healthy
for i in $(seq 1 20); do
  [ "$(docker inspect -f '{{.State.Health.Status}}' litellm_proxy)" = "healthy" ] && break
  sleep 3
done

# confirm the plugin loaded
docker exec litellm_proxy python3 -c \
  'import responses_tool_guard as m; print("OK", m.proxy_handler_instance is not None)'
```

## Functional test

Send a Responses-API request carrying the tools that previously caused the error:

```
curl -sN http://127.0.0.1:4000/v1/responses \
  -H 'content-type: application/json' \
  -H 'Authorization: Bearer $LITELLM_MASTER_KEY' \
  -d '{"model": "glm-5.1", "input": "hi", "tools": [{"type": "web_search"}]}'
```

**Expected:** HTTP **200** with a normal response stream — **no** **400 "validation error, 'function'"**.

## 7 Connect Codex

### 7.1 — Codex `config.toml`

On your workstation, edit `~/.codex/config.toml` (Windows: `%USERPROFILE%\codex\config.toml`):

```
model = "glm-5.1"
model_provider = "litellm"

[model_providers.litellm]
name = "LiteLLM"
base_url = "http://YOUR-SERVER-IP:4000/v1"
env_key = "LITELLM_API_KEY"
wire_api = "responses"
```

- `model` must match the **model group name** in `litellm_config.yaml` (e.g. `glm-5.1`).
- `wire_api = "responses"` uses the Responses API path that the guard plugin protects.
- For production, put LiteLLM behind HTTPS and use `base_url = "https://..."`.

### 7.2 — Set the API key

Use your LiteLLM master key (or a virtual key created via the `/key/generate` endpoint).

```
# macOS / Linux
export LITELLM_API_KEY="sk-your-litellm-master-key"

# Windows PowerShell – current session
$env:LITELLM_API_KEY = "sk-your-litellm-master-key"

# Windows PowerShell – persistent (survives reboots)
[Environment]::SetEnvironmentVariable("LITELLM_API_KEY","sk-your-litellm-master-key","User")
```

Then (re)start Codex Desktop and select the `glm-5.1` model. You're done.

## Troubleshooting

| Symptom                                         | Cause                                               | Fix                                                                            |
|-------------------------------------------------|-----------------------------------------------------|--------------------------------------------------------------------------------|
| <code>400 "validation error, 'function'"</code> | Non-function tools reaching Huawei                  | Plugin not loaded — check the callback is registered and the file is mounted.  |
| <code>404 APIG.0101 "API does not exist"</code> | LiteLLM forwarding <code>/responses</code> upstream | Add <code>use_chat_completions_api: true</code> to the model group.            |
| <code>401 auth_error</code>                     | Wrong/missing API key                               | Set <code>LITELLM_API_KEY</code> to a valid LiteLLM master/virtual key.        |
| Stream hangs / empty                            | Streaming-path mismatch                             | Ensure the <code>patches/</code> mounts are present and the image tag matches. |
| Model not found                                 | Wrong model name                                    | Use the <code>model_name</code> from the config, not the upstream model id.    |

## Rollback

```
cd /home/LiteLLM-Huawei-MaaS-Proxy
# restore backups (timestamp will vary)
cp docker-compose.yml.bak.* docker-compose.yml
cp assets/config/litellm_config.yaml.bak.* assets/config/litellm_config.yaml
rm assets/config/responses_tool_guard.py
docker compose up -d litellm
docker restart litellm_proxy
```

## Quick reference

| What                 | Value                                                                                                                  |
|----------------------|------------------------------------------------------------------------------------------------------------------------|
| LiteLLM proxy URL    | <code>http://SERVER-IP:4000</code>                                                                                     |
| Grafana              | <code>http://SERVER-IP:3000</code> (admin / <code>\$GRAFANA_PASSWORD</code> )                                          |
| Prometheus           | <code>http://SERVER-IP:9090</code>                                                                                     |
| Huawei MaaS endpoint | <code>https://api-ap-southeast-1.modelarts-maas.com/openai/v1</code>                                                   |
| LiteLLM image        | <code>ghcr.io/berriai/litellm:v1.83.14-stable.patch.3</code>                                                           |
| Plugin repo          | <code>github.com/Zefirox/litellm-responses-tool-guard</code>                                                           |
| Plugin metrics       | <code>rtg_stripped_tools_total</code> , <code>rtg_flattened_tools_total</code> ,<br><code>rtg_passthrough_total</code> |

Generated for the LiteLLM + Huawei MaaS + Codex deployment · Plugin: responses\_tool\_guard (Zefirox/litellm-responses-tool-guard) · LiteLLM v1.83.14-stable.patch.3